

RAG Copilot Week 1 单独执行手册

面向 Chenning + Codex 的每日代码任务拆解

项目：enterprise-rag-copilot | 阶段：Week 1 | 版本：v1.0 | 日期：2026-06-04

最高原则

这份文档不是普通学习计划，而是 RAG 项目第一周的工程执行说明。每天的任务必须同时让人看得懂、让 Codex 能执行、让后续 Review 能判断是否可以进入主线。AI 输出永远只是候选稿；真正完成标准是：接口清楚、边界稳定、测试或验收可复现、风险和失败记录可追踪。

读者	使用方式
Chenning	每天先读“今日目标”和“人工 Review Gate”，再把“给 Codex 的任务卡”复制给 Codex。Codex 输出后，不直接提交；先按验收命令和 Review Gate 检查。
Codex	只能在当天列出的文件范围内生成候选实现；必须先说明文件修改计划，再实现、补测试/验收、最后给 Review Checklist。
ChatGPT 后续对话	后续如果单独指导某一天任务，应以本文件为约束，不得跳过接口契约、边界检查、测试、失败记录和 commit 留痕。

0. 文档定位与执行范围

本文件只服务第一个项目：可迁移企业级 RAG Copilot。它从 Week 1 执行手册中抽取 RAG 相关任务，并按 Vibe Coding 项目实践公约重写成“每天可执行、可验收、可交给 Codex 的任务卡”。

- 本周主线：工程底座 + naive RAG v0 最小闭环。
- 项目名建议：enterprise-rag-copilot。
- 首个 demo domain：电商售后客服；但底层必须保留可迁移架构，不能写成单一电商客服脚本。
- 本文件不包含 RepoPilot、LeetCode、通用八股的每日任务；只保留与 RAG 项目直接相关的面试表达和工程记录。

第 1 周成功的定义：到 Day 7，项目能本地启动，/health 可访问，/chat 能走 data -> chunk -> index/fallback -> retrieve -> prompt -> answer，并返回 evidence list；同时有 README、architecture、API 契约、AI Coding workflow、失败记录和 commit 留痕。

1. Week 1 总目标与不做清单

方向	本周要达到的状态
工程底座	有清晰目录结构、FastAPI 入口、requirements.txt、.env.example、Dockerfile、README、基础 logging。
API 契约	GET /health 和 POST /chat 可运行；ChatRequest、ChatResponse、Evidence、TraceInfo 结构明确。
RAG v0	能加载电商 policy 文档，完成 chunking，建立最小 keyword/vector fallback index，retrieve top-k chunks，构造 prompt，生成 evidence-grounded answer。

可迁移边界	core pipeline 只处理通用 RAG 流程；domains/ecommerce 只处理电商 domain adapter、mock data、prompt/domain policy。
工程留痕	docs/design/architecture.md、docs/contracts/query_api.md、docs/ai-development-workflow.md、notes/failure_log.md、Week1 summary。
Codex 协作	每天有明确任务卡；Codex 输出必须通过人工 Review Gate 后才能 commit。

本周明确不做：SQL + RAG 完整混合问答、Redis、Locust 压测、RAGAS/DeepEval、rerank、复杂 LangGraph、完整 UI、真实生产部署、虚构性能数字。Week 1 的重点是工程骨架和最小闭环，不是包装指标。

2. 实践公约在本项目中的强制执行方式

2.1 任务区域判断

任务类型	区域	执行方式
临时试用某个 embedding API、向量库、prompt 写法	探索区	只能放在 experiments/；不能直接进入 app/ 主线。
FastAPI main、/chat、Pydantic schema、pipeline 接口	生产区雏形	先定义接口和错误语义，再写实现；必须有测试或验收命令。
document_loader、chunker、retriever、prompt_builder、answer_generator	生产区雏形	职责单一；不能跨层混写；不能把电商规则塞入 core pipeline。
README、architecture、AI workflow、failure log	生产区文档	必须写清楚边界、验收、风险、后续演化方向。

2.2 Codex 禁止事项

- 禁止一次性重写整个仓库；每天只改当天列出的文件。
- 禁止在 app/pipeline/ 中写死电商订单、退款、物流、保修等业务规则。
- 禁止让 retriever 直接生成最终答案；retriever 只返回 evidence/chunks。
- 禁止 answer_generator 编造 evidence 中不存在的信息。
- 禁止为了“能跑”随意新增复杂依赖；Week 1 优先使用最小依赖和 keyword fallback。
- 禁止删除或破坏已定义的 API 契约：ChatRequest、ChatResponse、Evidence、trace_id。
- 禁止只给代码不给验收命令；每次输出都要附 Review Checklist。
- 禁止把失败案例藏起来；失败要写入 notes/failure_log.md。

2.3 每天的固定交付物

交付物	最低要求
代码/文档	至少一个可 review 的小功能、测试、文档或数据文件。
验收记录	至少一组命令：pytest 或 curl；说明期望输出。
失败记录	记录 bug、依赖、API、检索、外部服务、设计卡点。
面试表达	写 3 句能解释今日设计取舍的话。
commit	至少 1 个 RAG 项目相关 commit；commit message 说明“做了什么 + 为什么”。

3. 推荐目录结构与模块边界

```
enterprise-rag-copilot/  
  app/  
    main.py  
    api/  
      routes.py  
  core/  
    config.py  
    logging_config.py  
  schemas/  
    chat.py  
    evidence.py  
    document.py  
    trace.py  
  pipeline/  
    document_loader.py  
    chunker.py  
    embedder.py  
    vector_store.py  
    retriever.py  
    prompt_builder.py  
    answer_generator.py  
    rag_pipeline.py  
  domains/  
    ecommerce/  
      adapter.py  
      mock_data.py  
      prompts.py  
  data/  
    ecommerce/  
      docs/  
        faq.md  
        return_policy.md  
        logistics_policy.md  
        warranty_policy.md  
      mock/  
        orders.json  
        products.json  
  docs/  
    design/  
      architecture.md  
    contracts/  
      query_api.md  
    ai-development-workflow.md  
    failure_cases.md  
  notes/  
    failure_log.md  
    week1_summary.md  
  tests/  
    test_health.py  
    test_chat_contract.py  
    test_document_loader.py
```

```
test_retriever.py
test_rag_pipeline.py
experiments/
README.md
requirements.txt
.env.example
Dockerfile
.dockerignore
```

模块	职责	不能做什么
api/routes.py	暴露 /health 和 /chat，调用 pipeline 或 service。	不能直接写检索、prompt、业务规则。
schemas/	定义请求、响应、Document、Chunk、Evidence、Trace 数据结构。	不能放复杂业务逻辑。
pipeline/	通用 RAG 流程：load、chunk、index、retrieve、prompt、answer。	不能写死 ecommerce 规则。
domains/ecommerce/	电商 domain adapter、mock data 读取、domain prompt 补充。	不能控制 FastAPI 入口或通用 pipeline。
data/ecommerce/	电商 demo 数据和政策文档。	不能放 Python 逻辑。
docs/	架构、API 契约、AI 协作规范、失败案例。	不能只写成功展示，也要写限制和 TODO。

4. API 契约：先固定接口，再实现功能

Week 1 的 /chat 可以先是 mock，再接 naive RAG pipeline，但响应结构从 Day 2 开始必须稳定。后续 Week 2 的 SQL tool、fallback、evaluation、performance tracing 都应挂在这个结构上，而不是推翻重写。

```
POST /chat
Request:
{
  "user_id": "demo_user_001",
  "session_id": "week1_demo_session",
  "query": "我买的耳机可以七天无理由退货吗？"
}

Response:
{
  "answer": "根据退货政策，符合条件的商品可在签收后 7 天内申请无理由退货...",
  "intent": "policy_question",
  "evidence": [
    {
      "source": "return_policy.md",
      "content": "签收后 7 天内，未拆封或不影响二次销售的商品可申请无理由退货。",
      "score": 0.82
    }
  ],
  "fallback": false,
  "fallback_reason": null,
  "trace_id": "trace_20260604_001"
}
```

- answer：必须基于 evidence；如果没有 evidence，就应该 fallback，而不是硬编。

- intent: Week 1 可以先用 simple heuristic 或 mock; Week 2 再升级 intent router。
- evidence: Week 1 最重要的工程字段之一, 为后续 citation correctness 和 faithfulness 评估打基础。
- fallback/fallback_reason: Week 1 可以简单实现检索为空 fallback; Week 2 再扩展低置信度、高风险、缺少订单号等情况。
- trace_id: Week 1 可以只生成 UUID; Week 2 再写入 tracing/performance。

5. Week 1 依赖策略与启动命令

为了避免第一周被复杂依赖拖住, 推荐先使用最小依赖。embedding 或向量库暂时不可用时, 必须有 keyword fallback, 保证项目主线始终能跑。

```
# requirements.txt 建议起步版本
fastapi
uvicorn[standard]
pydantic
pydantic-settings
python-dotenv
pytest
httpx

# 本地启动
python -m venv .venv
source .venv/bin/activate      # Windows 可使用 .venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload

# 验收
curl http://127.0.0.1:8000/health
curl -X POST http://127.0.0.1:8000/chat \
  -H "Content-Type: application/json" \
  -d '{"user_id":"u1","session_id":"s1","query":"退货政策是什么?"}'
pytest
```

Day 1: 工程初始化 + /health + 最小工程骨架

今日目标：把项目从脚本式想法变成可启动的 FastAPI 工程骨架。今天不做 RAG 逻辑，只建立可以长期维护的项目入口。

A. 今天要创建/修改的文件

- README.md
- requirements.txt
- .env.example
- Dockerfile
- app/main.py
- app/api/routes.py
- app/core/config.py
- tests/test_health.py

B. 给 Codex 的任务卡

Context:

你正在 enterprise-rag-copilot 仓库中工作。本任务属于 RAG 项目 Week 1 的生产区雏形代码，不是一次性 toy demo。必须遵守 core pipeline 与 domains/ecommerce 的边界。

Goal:

创建 enterprise-rag-copilot 的最小 FastAPI 工程，提供 GET /health，并让 pytest 能验证服务健康检查。

Constraints:

- 不要修改当天未列出的文件，除非先说明必要性。
- 不要新增复杂依赖；如确需新增，先解释原因和替代方案。
- app/pipeline/ 保持通用，不写死电商规则。
- domains/ecommerce/ 只放 domain adapter、mock data 读取、domain prompt。
- 新增接口必须有测试或 curl 验收命令。
- 输出代码后必须给 Review Checklist。

Files:

- README.md
- requirements.txt
- .env.example
- Dockerfile
- app/main.py
- app/api/routes.py
- app/core/config.py
- tests/test_health.py

Process:

1. 先列出你准备修改的文件和原因。
2. 再给出最小实现，不要一次性扩散到无关模块。
3. 为新增功能补充至少一个测试或手动验收命令。
4. 最后输出 Review Checklist，说明风险、临时策略和后续 TODO。

Review:

必须通过今天的验收命令；不能破坏已有 /health、/chat 契约；不能把电商业务规则写入 core pipeline。

C. 实现步骤

1. 创建仓库根目录与 app/api/core/tests 基础结构。
2. 在 app/main.py 中创建 FastAPI app，并 include router。
3. 在 app/api/routes.py 中实现 GET /health，返回 status、service、version。
4. 在 app/core/config.py 中集中放 service name、version、environment 等配置。
5. 写 tests/test_health.py，用 TestClient 验证 /health。
6. 写 README 初版：项目定位、不是 toy demo、Week 1 目标、启动命令。
7. 创建 .env.example 和 Dockerfile 最小版。

D. 验收命令

```
pip install -r requirements.txt
uvicorn app.main:app --reload
curl http://127.0.0.1:8000/health
pytest tests/test_health.py
```

E. 人工 Review Gate

- /health 的返回结构稳定，不是随便返回字符串。
- README 能解释项目定位：transferable enterprise RAG Copilot，而不是电商客服脚本。
- FastAPI app、router、config 职责分离。
- 没有引入 RAG 逻辑或电商业务规则。
- pytest 能通过。

F. 建议 commit message

```
git add . && git commit -m "chore: initialize rag copilot fastapi skeleton"
git commit -m "test: add health check test"
```

G. 今日失败记录

记录今天是否遇到 Python 环境、依赖安装、uvicorn 启动、路径导入、测试客户端问题。写清楚 Input / Expected / Actual / Fix。

H. 今日三句面试表达

- 我先把 RAG 项目搭成 FastAPI 服务，而不是直接写成脚本，这样后续 API、评估和部署都可以自然接入。
- 我把配置、路由和入口拆开，是为了避免后续所有逻辑挤在 main.py 里。
- 第一天的目标不是实现 RAG，而是建立可以继续迭代和验收的工程骨架。

Day 2: Pydantic API 契约 + /chat mock

今日目标：先固定 /chat 的请求和响应结构，让后续 retrieval、evidence、fallback、trace 都能稳定挂上去。

A. 今天要创建/修改的文件

- app/schemas/chat.py
- app/schemas/evidence.py
- app/schemas/trace.py
- app/api/routes.py
- docs/contracts/query_api.md
- tests/test_chat_contract.py

B. 给 Codex 的任务卡

Context:

你正在 enterprise-rag-copilot 仓库中工作。本任务属于 RAG 项目 Week 1 的生产区雏形代码，不是一次性 toy demo。必须遵守 core pipeline 与 domains/ecommerce 的边界。

Goal:

定义 ChatRequest、ChatResponse、Evidence、TraceInfo，并实现 POST /chat mock，返回结构化响应。

Constraints:

- 不要修改当天未列出的文件，除非先说明必要性。
- 不要新增复杂依赖；如确需新增，先解释原因和替代方案。
- app/pipeline/ 保持通用，不写死电商规则。
- domains/ecommerce/ 只放 domain adapter、mock data 读取、domain prompt。
- 新增接口必须有测试或 curl 验收命令。
- 输出代码后必须给 Review Checklist。

Files:

- app/schemas/chat.py
- app/schemas/evidence.py
- app/schemas/trace.py
- app/api/routes.py
- docs/contracts/query_api.md
- tests/test_chat_contract.py

Process:

1. 先列出你准备修改的文件和原因。
2. 再给出最小实现，不要一次性扩散到无关模块。
3. 为新增功能补充至少一个测试或手动验收命令。
4. 最后输出 Review Checklist，说明风险、临时策略和后续 TODO。

Review:

必须通过今天的验收命令；不能破坏已有 /health、/chat 契约；不能把电商业务规则写入 core pipeline。

C. 实现步骤

1. 创建 schemas/evidence.py，定义 Evidence(source, content, score)。
2. 创建 schemas/trace.py，定义 TraceInfo 或至少 trace_id 字段规范。

3. 创建 schemas/chat.py, 定义 ChatRequest(user_id, session_id, query) 和 ChatResponse(answer, intent, evidence, fallback, fallback_reason, trace_id)。
4. 在 /chat 中先返回 mock answer, 但必须符合 ChatResponse。
5. 对空 query 或过短 query 进行基础校验。
6. 写 docs/contracts/query_api.md, 记录 request/response JSON 示例和字段含义。
7. 写 tests/test_chat_contract.py, 验证成功请求和非法请求。

D. 验收命令

```
uvicorn app.main:app --reload
curl -X POST http://127.0.0.1:8000/chat -H "Content-Type: application/json" -d '{"user_id":"u1","session_id":"s1","query":"退货政策是什么?"}'
pytest tests/test_chat_contract.py
```

E. 人工 Review Gate

- ChatResponse 不是只有 answer, 一定要保留 evidence、fallback、trace_id。
- 字段命名清楚, 后续 Week 2 不需要推翻重写。
- 非法输入要有清晰错误, 而不是服务器内部报错。
- docs/contracts/query_api.md 与代码字段一致。
- mock 逻辑不混入真正 retrieval。

F. 建议 commit message

```
git commit -m "feat: define chat api contract"
git commit -m "docs: add query api contract"
```

G. 今日失败记录

记录今天 API 契约设计中最难决定的字段, 以及为什么保留 evidence/fallback/trace_id。

H. 今日三句面试表达

- 我没有让 /chat 只返回纯文本, 而是从第一版开始返回结构化响应。
- evidence、fallback 和 trace_id 是后续做质量评估、错误分析和性能追踪的基础字段。
- 先定 API 契约可以减少后续反复改接口造成的技术债。

Day 3: 电商 demo 最小数据 + 文档加载层

今日目标：准备电商售后 demo 的最小数据源，并实现通用 document loader。今天重点是“数据进入系统”，不是检索质量。

A. 今天要创建/修改的文件

- data/ecommerce/mock/orders.json
- data/ecommerce/mock/products.json
- data/ecommerce/docs/faq.md
- data/ecommerce/docs/return_policy.md
- data/ecommerce/docs/logistics_policy.md
- data/ecommerce/docs/warranty_policy.md
- app/schemas/document.py
- app/pipeline/document_loader.py
- app/domains/ecommerce/adapter.py
- tests/test_document_loader.py

B. 给 Codex 的任务卡

Context:

你正在 enterprise-rag-copilot 仓库中工作。本任务属于 RAG 项目 Week 1 的生产区雏形代码，不是一次性 toy demo。必须遵守 core pipeline 与 domains/ecommerce 的边界。

Goal:

创建最小电商 mock 数据和 policy 文档，实现可以读取 markdown policy docs 的 document loader。

Constraints:

- 不要修改当天未列出的文件，除非先说明必要性。
- 不要新增复杂依赖；如确需新增，先解释原因和替代方案。
- app/pipeline/ 保持通用，不写死电商规则。
- domains/ecommerce/ 只放 domain adapter、mock data 读取、domain prompt。
- 新增接口必须有测试或 curl 验收命令。
- 输出代码后必须给 Review Checklist。

Files:

- data/ecommerce/mock/orders.json
- data/ecommerce/mock/products.json
- data/ecommerce/docs/faq.md
- data/ecommerce/docs/return_policy.md
- data/ecommerce/docs/logistics_policy.md
- data/ecommerce/docs/warranty_policy.md
- app/schemas/document.py
- app/pipeline/document_loader.py
- app/domains/ecommerce/adapter.py
- tests/test_document_loader.py

Process:

1. 先列出你准备修改的文件和原因。
2. 再给出最小实现，不要一次性扩散到无关模块。
3. 为新增功能补充至少一个测试或手动验收命令。
4. 最后输出 Review Checklist，说明风险、临时策略和后续 TODO。

Review:

必须通过今天的验收命令；不能破坏已有 /health、/chat 契约；不能把电商业务规则写入 core pipeline。

C. 实现步骤

1. 创建 5-10 条 orders/products mock 数据，字段保持简单但真实：order_id、user_id、product_id、status、created_at、refund_status。
2. 创建 4 类 policy 文档：FAQ、退货政策、物流政策、保修政策，每类至少 2-3 条规则。
3. 定义 Document(id, source, content, metadata) 数据结构。
4. 实现 load_policy_docs(data_dir) 或 load_documents(path)，读取 markdown 文件并输出 Document list。
5. 在 domains/ecommerce/adapters.py 中封装 get_policy_doc_paths 或 load_ecommerce_documents，保持 domain 入口清楚。
6. 写测试验证：能读取 4 个 policy 文档，metadata 包含 document_type/source。

D. 验收命令

```
pytest tests/test_document_loader.py
python -m pytest tests/test_document_loader.py -q
```

E. 人工 Review Gate

- mock 数据只能放在 data/ecommerce，不要写进 pipeline。
- document_loader 应能复用于其他 domain，不应只识别电商文件名。
- Document 结构要有 metadata，给 Week 2 metadata filtering 留空间。
- policy 文档内容可以少，但必须足以支持 Day 5 /chat demo。
- 测试覆盖文档存在和 metadata 正确。

F. 建议 commit message

```
git commit -m "data: add ecommerce demo documents and mock data"
git commit -m "feat: implement document loader"
```

G. 今日失败记录

记录今天数据设计里哪些字段是为了 Week 2 SQL tool / metadata filtering 预留的，哪些只是 Week 1 demo 所需。

H. 今日三句面试表达

- 我把电商客服数据拆成结构化 mock 数据和非结构化 policy 文档两类，为 Week 2 的 SQL + RAG 混合问答做准备。
- 订单状态这类精确事实后续适合走结构化工具，而退货/物流/保修政策更适合走文档检索。
- document loader 保持通用，不绑定电商业务规则，这样后续可以替换成 HR 或 IT support domain。

Day 4: Chunking + Index + Keyword Fallback

今日目标：实现从 Document 到 Chunk，再到最小可检索索引的链路。真实 embedding 不稳定时，keyword fallback 必须能保证系统可运行。

A. 今天要创建/修改的文件

- app/schemas/document.py
- app/pipeline/chunker.py
- app/pipeline/embedder.py
- app/pipeline/vector_store.py
- app/pipeline/retriever.py
- tests/test_retriever.py
- experiments/README.md

B. 给 Codex 的任务卡

Context:

你正在 enterprise-rag-copilot 仓库中工作。本任务属于 RAG 项目 Week 1 的生产区雏形代码，不是一次性 toy demo。必须遵守 core pipeline 与 domains/ecommerce 的边界。

Goal:

实现 split_documents、build_index 和 retrieve 的最小版本；优先保证 keyword fallback 可测试、可运行。

Constraints:

- 不要修改当天未列出的文件，除非先说明必要性。
- 不要新增复杂依赖；如确需新增，先解释原因和替代方案。
- app/pipeline/ 保持通用，不写死电商规则。
- domains/ecommerce/ 只放 domain adapter、mock data 读取、domain prompt。
- 新增接口必须有测试或 curl 验收命令。
- 输出代码后必须给 Review Checklist。
- 不要因为 embedding API 未配置就让主线不可运行。
- 如果写 mock embedder，必须标记为 mock/fallback，不得伪装成真实性能。

Files:

- app/schemas/document.py
- app/pipeline/chunker.py
- app/pipeline/embedder.py
- app/pipeline/vector_store.py
- app/pipeline/retriever.py
- tests/test_retriever.py
- experiments/README.md

Process:

1. 先列出你准备修改的文件和原因。
2. 再给出最小实现，不要一次性扩散到无关模块。
3. 为新增功能补充至少一个测试或手动验收命令。
4. 最后输出 Review Checklist，说明风险、临时策略和后续 TODO。

Review:

必须通过今天的验收命令；不能破坏已有 /health、/chat 契约；不能把电商业务规则写入 core pipeline。

C. 实现步骤

1. 扩展 document schema: 定义 Chunk(id, document_id, source, content, metadata)。
2. 实现 split_documents(documents, chunk_size, overlap), 先用简单字符或段落切分。
3. 实现 embedder.py: 可以先提供 MockEmbedder 或 KeywordEmbedder, 保证 deterministic。
4. 实现 vector_store.py: InMemoryIndex, 保存 chunks 和可搜索表示。
5. 实现 retriever.py: retrieve(query, top_k), 返回 Evidence-like chunks 和 score。
6. 若要试用真实 embedding, 把实验放到 experiments/, 主线仍使用 fallback。
7. 写 tests/test_retriever.py: 查询“退货”能返回 return_policy 相关 chunk。

D. 验收命令

```
pytest tests/test_retriever.py
python -m pytest tests/test_document_loader.py tests/test_retriever.py -q
```

E. 人工 Review Gate

- chunker、embedder、vector_store、retriever 职责清楚, 不互相越界。
- retriever 只返回 chunks/evidence, 不生成 answer。
- score 的含义即使简单, 也要在注释或文档里说明。
- fallback 明确标记, 不能虚称“向量检索效果很好”。
- 至少一个查询能稳定返回相关 policy。

F. 建议 commit message

```
git commit -m "feat: add chunking and in-memory retrieval fallback"
git commit -m "test: cover ecommerce policy retrieval"
```

G. 今日失败记录

记录今天检索失败案例: 哪个 query 找不到正确文档? 原因是 chunk 太粗、关键词不匹配、数据太少, 还是 score 规则太弱?

H. 今日三句面试表达

- 我实现了从文档加载、切分到最小检索的 RAG 数据链路。
- 我保留 keyword fallback, 避免系统因为外部 embedding 服务不可用而完全无法演示。
- 第一版 retrieval 追求可运行和可测试, 不虚构高级向量检索质量。

Day 5: Retrieve -> Prompt -> Answer 最小闭环

今日目标：把 /chat 从 mock 接口接到 naive RAG pipeline，让用户问题能经过检索、prompt 构造和 evidence-grounded answer 返回。

A. 今天要创建/修改的文件

- app/pipeline/prompt_builder.py
- app/pipeline/answer_generator.py
- app/pipeline/rag_pipeline.py
- app/api/routes.py
- app/schemas/chat.py
- tests/test_rag_pipeline.py
- docs/failure_cases.md

B. 给 Codex 的任务卡

Context:

你正在 enterprise-rag-copilot 仓库中工作。本任务属于 RAG 项目 Week 1 的生产区雏形代码，不是一次性 toy demo。必须遵守 core pipeline 与 domains/ecommerce 的边界。

Goal:

实现 naive RAG pipeline: query -> retrieve -> build_prompt -> generate_answer -> ChatResponse, 并返回 evidence list。

Constraints:

- 不要修改当天未列出的文件，除非先说明必要性。
- 不要新增复杂依赖；如确需新增，先解释原因和替代方案。
- app/pipeline/ 保持通用，不写死电商规则。
- domains/ecommerce/ 只放 domain adapter、mock data 读取、domain prompt。
- 新增接口必须有测试或 curl 验收命令。
- 输出代码后必须给 Review Checklist。
- answer_generator 只能基于 retrieved chunks 组织回答；检索为空必须 fallback。
- 不要在 prompt_builder 中写死某个具体问题的答案。

Files:

- app/pipeline/prompt_builder.py
- app/pipeline/answer_generator.py
- app/pipeline/rag_pipeline.py
- app/api/routes.py
- app/schemas/chat.py
- tests/test_rag_pipeline.py
- docs/failure_cases.md

Process:

1. 先列出你准备修改的文件和原因。
2. 再给出最小实现，不要一次性扩散到无关模块。
3. 为新增功能补充至少一个测试或手动验收命令。
4. 最后输出 Review Checklist，说明风险、临时策略和后续 TODO。

Review:

必须通过今天的验收命令；不能破坏已有 /health、/chat 契约；不能把电商业务规则写入 core pipeline。

C. 实现步骤

1. 实现 `build_prompt(query, chunks)`: `prompt` 必须包含用户问题和检索片段。
2. 实现 `generate_answer(prompt, evidence)`: Week 1 可以是 mock LLM / rule-based answer, 但必须只引用 `evidence`。
3. 实现 `rag_pipeline.py`: 封装 `run_chat(query, user_id, session_id)` 返回 `ChatResponse`。
4. 修改 `/chat`: 调用 `rag_pipeline`, 而不是返回固定 mock。
5. 当 `retrieve` 为空或 `score` 过低时, 返回 `fallback=true` 和 `fallback_reason`。
6. 写 `tests/test_rag_pipeline.py`: 一个成功 policy query, 一个 unknown/fallback query。
7. 在 `docs/failure_cases.md` 记录至少 2 个当前 v0 局限。

D. 验收命令

```
uvicorn app.main:app --reload
curl -X POST http://127.0.0.1:8000/chat -H "Content-Type: application/json" -d '{"user_id":"u1","session_id":"s1","query":"耳机可以退货吗?"}'
curl -X POST http://127.0.0.1:8000/chat -H "Content-Type: application/json" -d '{"user_id":"u1","session_id":"s1","query":"火星仓库配送政策是什么?"}'
pytest tests/test_rag_pipeline.py
```

E. 人工 Review Gate

- `/chat` 返回 `answer + evidence`, 不是只有 `answer`。
- 成功 query 的 `evidence source` 能追溯到 policy 文件。
- unknown query 不硬编, 能 `fallback`。
- `prompt_builder` 不调用 `retriever`; `answer_generator` 不调用 `data loader`。
- `docs/failure_cases.md` 不只写成功案例, 也写 v0 的限制。

F. 建议 commit message

```
git commit -m "feat: connect chat endpoint to naive rag pipeline"
git commit -m "docs: record week1 rag failure cases"
```

G. 今日失败记录

记录今天最典型的回答失败: 是 `evidence` 不相关、`answer` 太模板化、`fallback` 误触发, 还是 query 没有被正确检索?

H. 今日三句面试表达

- 我跑通了 naive RAG v0: 用户问题、文档检索、prompt 构造、回答生成和 `evidence` 返回。
- 回答带 `evidence` 是为了后续评估 `citation correctness` 和 `answer faithfulness`。
- 当检索不到证据时, 系统会 `fallback`, 而不是让模型凭空编造。

Day 6: Docker + Logging + 架构文档 + AI Coding Workflow

今日目标：补齐工程项目最容易被忽略的部分：可复现启动、日志、架构说明、AI 协作规范。今天不堆新功能。

A. 今天要创建/修改的文件

- Dockerfile
- .dockerignore
- .env.example
- app/core/logging_config.py
- app/main.py
- app/pipeline/rag_pipeline.py
- docs/design/architecture.md
- docs/ai-development-workflow.md
- README.md

B. 给 Codex 的任务卡

Context:

你正在 enterprise-rag-copilot 仓库中工作。本任务属于 RAG 项目 Week 1 的生产区雏形代码，不是一次性 toy demo。必须遵守 core pipeline 与 domains/ecommerce 的边界。

Goal:

完善 Dockerfile、基础 logging、architecture 文档和 AI Coding workflow，使项目具备可复现和可解释的工程痕迹。

Constraints:

- 不要修改当天未列出的文件，除非先说明必要性。
- 不要新增复杂依赖；如确需新增，先解释原因和替代方案。
- app/pipeline/ 保持通用，不写死电商规则。
- domains/ecommerce/ 只放 domain adapter、mock data 读取、domain prompt。
- 新增接口必须有测试或 curl 验收命令。
- 输出代码后必须给 Review Checklist。
- 不要为了 Docker 引入复杂 Compose；Week 1 只做单服务 Dockerfile。
- logging 记录关键阶段即可，不要输出敏感数据或完整 prompt。

Files:

- Dockerfile
- .dockerignore
- .env.example
- app/core/logging_config.py
- app/main.py
- app/pipeline/rag_pipeline.py
- docs/design/architecture.md
- docs/ai-development-workflow.md
- README.md

Process:

1. 先列出你准备修改的文件和原因。
2. 再给出最小实现，不要一次性扩散到无关模块。
3. 为新增功能补充至少一个测试或手动验收命令。
4. 最后输出 Review Checklist，说明风险、临时策略和后续 TODO。

Review:

必须通过今天的验收命令；不能破坏已有 /health、/chat 契约；不能把电商业务规则写入 core pipeline。

C. 实现步骤

1. 补 Dockerfile：能安装依赖并启动 uvicorn。
2. 补 .dockerignore：排除 .venv、__pycache__、.pytest_cache、.git 等。
3. 实现 logging_config.py，并在 app 启动和 RAG pipeline 关键阶段记录 trace_id、stage、latency placeholder。
4. 写 docs/design/architecture.md：项目定位、core pipeline、domain adapter、v0 能力、Week2 演化计划。
5. 写 docs/ai-development-workflow.md：如何使用 Codex、哪些必须人工 review、测试和 commit 规则。
6. README 补充 Quick Start、API Example、Project Structure、Week1 Progress。

D. 验收命令

```
docker build -t enterprise-rag-copilot:week1 .
docker run --rm -p 8000:8000 enterprise-rag-copilot:week1
curl http://127.0.0.1:8000/health
pytest
```

E. 人工 Review Gate

- Dockerfile 能启动项目，不只是占位文件。
- README 中的命令复制后能运行。
- architecture.md 明确 core pipeline 与 ecommerce adapter 的边界。
- AI workflow 文档明确：Codex 输出是候选稿，必须 review 后提交。
- logging 不用 print 代替，且不泄露敏感信息。

F. 建议 commit message

```
git commit -m "chore: add dockerfile and logging setup"
git commit -m "docs: add rag architecture and ai development workflow"
```

G. 今日失败记录

记录今天是否遇到 Docker build/run 问题、环境变量问题、日志过多/过少问题，以及 README 命令是否真正可复现。

H. 今日三句面试表达

- 我为项目补充 Dockerfile 和 .env.example，让别人可以更容易复现我的运行环境。
- 我用 logging 记录 pipeline 关键阶段，而不是依赖临时 print。
- 我把 AI Coding workflow 写进文档，说明 AI 负责辅助生成，设计、review、测试和合并由我控制。

Day 7: 本周收尾 + 全链路验收 + README/总结

今日目标：不再新增大功能，而是检查 Week 1 的 RAG v0 是否真正可运行、可解释、可继续迭代。

A. 今天要创建/修改的文件

- README.md
- docs/design/architecture.md
- docs/contracts/query_api.md
- docs/failure_cases.md
- notes/failure_log.md
- notes/week1_summary.md
- tests/

B. 给 Codex 的任务卡

Context:

你正在 enterprise-rag-copilot 仓库中工作。本任务属于 RAG 项目 Week 1 的生产区雏形代码，不是一次性 toy demo。必须遵守 core pipeline 与 domains/ecommerce 的边界。

Goal:

完成 Week 1 收尾：全量测试、手动 API 验收、README 更新、failure log 和 week1 summary。

Constraints:

- 不要修改当天未列出的文件，除非先说明必要性。
- 不要新增复杂依赖；如确需新增，先解释原因和替代方案。
- app/pipeline/ 保持通用，不写死电商规则。
- domains/ecommerce/ 只放 domain adapter、mock data 读取、domain prompt。
- 新增接口必须有测试或 curl 验收命令。
- 输出代码后必须给 Review Checklist。
- 今天不新增复杂功能；只修复阻塞验收的问题。
- 不允许为了通过验收删除测试或降低接口约束。

Files:

- README.md
- docs/design/architecture.md
- docs/contracts/query_api.md
- docs/failure_cases.md
- notes/failure_log.md
- notes/week1_summary.md
- tests/

Process:

1. 先列出你准备修改的文件和原因。
2. 再给出最小实现，不要一次性扩散到无关模块。
3. 为新增功能补充至少一个测试或手动验收命令。
4. 最后输出 Review Checklist，说明风险、临时策略和后续 TODO。

Review:

必须通过今天的验收命令；不能破坏已有 /health、/chat 契约；不能把电商业务规则写入 core pipeline。

C. 实现步骤

1. 运行全部测试，修复失败测试。
2. 手动 curl /health 和 /chat，保存 2 个示例请求与响应到 README 或 docs。
3. 检查 README：Motivation、Architecture、Quick Start、API Examples、Week1 Progress、Next Steps。
4. 完善 docs/failure_cases.md：至少记录 3 个 v0 局限和 Week2 对应改进。
5. 整理 notes/failure_log.md：按日期记录 bug 和修复。
6. 写 notes/week1_summary.md：完成内容、设计取舍、未完成事项、Week2 plan。
7. 确认至少 5 条 RAG 相关 commit。

D. 验收命令

```
pytest
curl http://127.0.0.1:8000/health
curl -X POST http://127.0.0.1:8000/chat -H "Content-Type: application/json" -d '{"user_id":"u1","session_id":"s1","query":"退货政策是什么?"}'
git log --oneline --decorate -n 10
```

E. 人工 Review Gate

- 项目能从 README 复制命令启动。
- /chat 返回 answer、evidence、fallback、trace_id。
- 所有测试通过；如果有未解决问题，必须写明原因和影响范围。
- failure_cases 真实记录局限，不粉饰 v0。
- Week2 plan 能自然承接 SQL tool、metadata、evidence、fallback、evaluation、performance tracing。

F. 建议 commit message

```
git commit -m "docs: finalize week1 rag progress and next steps"
git commit -m "test: verify week1 rag v0 end-to-end"
```

G. 今日失败记录

写 Week1 最大失败/卡点复盘：什么问题最影响推进？是环境、接口、数据、检索、文档还是 AI 输出质量？下周如何避免？

H. 今日三句面试表达

- 第一周我完成了企业 RAG Copilot 的工程底座，并跑通了 naive RAG v0。
- 我没有把项目写成固定电商客服脚本，而是保留 core pipeline + domain adapter 的可迁移结构。
- Week 2 我会把 v0 升级为 SQL + 文档检索混合流程，并补 evidence citation、fallback、evaluation 和 performance tracing。

6. 每天交给 Codex 前的通用 Prompt 模板

每天使用 Codex 前，先把当天任务卡复制进去，再追加下面这段总约束。这样可以减少 Codex 乱改文件、乱加依赖、把 toy demo 代码塞进主线的问题。

你是 enterprise-rag-copilot 项目的编码助手。请严格遵守：

1. 先输出修改计划，不要直接大段改代码。

- 2. 只修改当天任务卡列出的文件；如必须新增文件，先解释原因。
- 3. app/pipeline 是通用 RAG pipeline，禁止写死电商业务规则。
- 4. domains/ecommerce 才能放电商 domain adapter、mock data 读取、domain prompt。
- 5. 新增代码要清楚、可读、学生/工程版写法，不要炫技，不要一行式复杂写法。
- 6. 每个功能都要给测试或手动验收命令。
- 7. 输出末尾必须包含：Modified Files、How to Verify、Review Checklist、Known Risks、Follow-up TODO。
- 8. 你的代码只是候选实现，我会人工 review 后再提交。

7. 人工 Review Checklist

维度	检查问题	拒绝合并信号
Correctness	是否真正完成当天目标？ /health、 /chat、 tests 是否通过？	接口跑不通；测试失败；空 query 崩溃。
Boundary	core pipeline 是否保持通用？ domain 规则是否只在 ecommerce adapter/data 中？	pipeline 里写死退货、订单、物流规则。
Evidence	answer 是否能追溯到 evidence？检索为空是否 fallback？	没有 evidence 也强答；answer 编造不存在内容。
Readability	函数职责是否单一？命名是否清楚？控制流是否容易读？	一个函数同时加载数据、检索、生成回答、处理 API。
Maintainability	是否引入不必要依赖？是否破坏已有 schema？	为了 demo 加复杂库；随意改 ChatResponse 字段。
Testability	是否有 pytest 或 curl 验收？失败路径是否可测？	只能手动看起来能跑；无失败 case。
Traceability	是否有 trace_id、failure log、commit message？	没有记录失败，也无法定位一次请求。

8. 失败记录模板

```
# notes/failure_log.md

## Day X - YYYY-MM-DD

### Bug / Failure Summary
一句话描述今天遇到的问题。

### Input
触发问题的命令、请求、query 或测试。

### Expected
期望发生什么。

### Actual
实际发生什么，包含错误信息或异常输出。

### Hypothesis
我怀疑原因是什么。

### Fix / Decision
今天如何修复；如果未修复，为什么暂缓。

### Test Added / Verification
```

新增了什么测试或用了什么命令验证。

Interview Note
这个失败如何转化成面试中能讲的工程经验。

9. Week 1 最终验收清单

类别	验收项
工程底座	README.md 可指导别人启动项目 requirements.txt 可安装 .env.example 存在 Dockerfile 可 build/run logging_config.py 存在并被使用
API	GET /health 可访问 POST /chat 可访问 ChatRequest/ChatResponse/Evidence/Trace 字段清楚 docs/contracts/query_api.md 与代码一致
数据	有 orders/products mock data 有 FAQ/return/logistics/warranty policy docs 文档 loader 能读取 markdown Document metadata 可用
RAG v0	能 chunk 能 build keyword/vector fallback index 能 retrieve top-k chunks 能 build prompt 能 generate evidence-grounded answer 检索为空能 fallback
文档	architecture.md 说明 core/domain/infrastructure ai-development-workflow.md 说明 Codex 使用规范 failure_cases.md 记录 v0 局限 week1_summary.md 完成
实践公约	Codex 输出经过人工 review 每天有 failure log 至少 5 条 RAG commit 没有虚构性能指标 没有把电商规则写死进 core pipeline

10. Week 2 承接方向

Week 1 结束后，不要马上重写项目。Week 2 应在现有接口和模块边界上升级，而不是推翻。优先顺序如下：

1. 把 orders/products/refunds 从 JSON mock 升级为 SQLite 或更清晰的 structured tool interface。
2. 实现 intent router：订单状态、物流、退款、退换货政策、商品保修、混合问题。
3. 补 metadata filtering：document_type、product_category、policy_version、applicable_scenario。
4. 升级 evidence builder：数据库结果和文档片段都能进入 evidence list。
5. 扩展 fallback handler：低分检索、缺少订单号、高风险售后场景。
6. 加入 evaluation cases：至少 30 条，覆盖结构化、文档、混合、失败问题。
7. 加入 performance tracer：记录 intent、SQL、retrieval、LLM/mock LLM、total latency。

Week 1 最重要的一句话：不是证明你已经懂企业级 RAG 的全部技术，而是证明你能把一个 RAG 想法变成可运行、可解释、可测试、可继续迭代的工程骨架。